

**POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY**

Faculty of Information Technology

Intelligent System Development

ASSIGNMENT 06

**Recurrent Neural Networks on Time Series:
From Scratch, in Keras and in PyTorch**

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Nguyễn Văn Trường
Student ID: B23DCCE095
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1	Executive Summary	2
2	RNN Fundamentals	3
2.1	Sequences as data	3
2.2	Operators	3
2.3	The recurrent step	3
2.4	Back-propagation through time	3
2.5	Vanishing and exploding gradients, clipping	4
2.6	LSTM and GRU	4
2.7	One layer, three names	4
3	Datasets and Preprocessing	4
3.1	EC: e-commerce service process (Online Retail II)	5
3.2	ST: S&P 500 stock prices	5
3.3	Splits, scaling and metrics	6
4	Models and Fairness Rules	6
5	Implementations	7
5.1	Scratch (NumPy)	7
5.2	Keras	7
5.3	PyTorch	8
5.4	Equivalence check	8
6	Results on EC (e-commerce service process)	8
7	Results on ST (stock returns)	10
8	Cross-Dataset Discussion	11
9	Conclusion and Limitations	11

1 Executive Summary

This assignment studies the recurrent neural network (RNN) on two time-related datasets and implements the same model three times: **from scratch** in NumPy (**SC**, hand-written forward pass, back-propagation through time and Adam), in **Keras** (**TF**) and in **PyTorch** (**PT**). The datasets are **EC**, a service process of an e-commerce company (Online Retail II: orders and cancellations of 750 products over 604 open days, 432,000 windows) and **ST**, stock prices (S&P 500: 350 stocks over 1,258 trading days, 429,800 windows). Both tasks are many-to-one regression: a window of past days predicts the next day. All implementations share the data, the split, the architecture (one vanilla RNN layer with 32 units and a linear head), the starting weights, the optimizer and the early-stopping rule, so the only thing that changes is the abstraction level. LSTM and GRU are added as extras in Keras and PyTorch. Run IDs have the form `<dataset>-<implementation>-<model>`, for example `ST-TF-GRU`.

Table 1: All 14 runs, test period. RMSE in units sold (EC, top block) and in log-return (ST, bottom block); R^2 is out-of-sample against the train-mean predictor; s/epoch = median time per epoch on CPU.

Run	test RMSE	R^2	params	s/epoch
EC-SC-RNN	62.51	0.439	1,409	3.9
EC-TF-RNN	63.01	0.438	1,409	6.2
EC-PT-RNN	62.84	0.439	1,409	11.2
EC-TF-LSTM	62.44	0.446	5,537	14.6
EC-PT-LSTM	62.03	0.447	5,665	22.0
EC-TF-GRU	62.41	0.449	4,257	16.4
EC-PT-GRU	62.24	0.448	4,257	28.4
ST-SC-RNN	0.0159	-0.017	1,217	4.2
ST-TF-RNN	0.0159	-0.017	1,217	6.5
ST-PT-RNN	0.0158	-0.016	1,217	11.8
ST-TF-LSTM	0.0158	-0.014	4,769	14.9
ST-PT-LSTM	0.0158	-0.012	4,897	20.3
ST-TF-GRU	0.0158	-0.015	3,681	17.0
ST-PT-GRU	0.0158	-0.013	3,681	29.7

Key takeaways

1. **The three implementations compute the same function.** With identical weights the forward passes differ by less than $6 \cdot 10^{-7}$ and the parameter counts match exactly. After training, the core RNN on EC ends at RMSE 62.51 / 63.01 / 62.84 (scratch / Keras / PyTorch), a 0.8 % spread caused by batch order and the epoch at which early stopping fires, not by different mathematics. On ST the three core RNNs are indistinguishable (RMSE 0.0159, 0.0159, 0.0158).
2. **On EC the RNN learns real structure.** All seven recurrent models reach RMSE 62.0–63.0 against 67.9 for the train mean, 81.1 for a seasonal-naive forecast and 82.7 for persistence ($R^2 \approx 0.44$).
3. **On ST nothing beats the trivial forecast.** RMSE is 0.0158–0.0159 versus 0.0157 for predicting zero return; R^2 is slightly negative and directional accuracy (48–51 %) is below the 53.1 % of always predicting “up”. This is the expected, honest outcome for daily returns, which are close to a random walk.
4. **Gates help little here and cost a lot.** LSTM/GRU improve EC by 0.5–1.5 % and do not help on ST, while costing 2–3.5× the time per epoch and 3–4× the parameters.
5. **Cost.** For the vanilla RNN on CPU, the NumPy loop (3.9–4.2 s/epoch) is faster than Keras

(6.2–6.5 s) and PyTorch (11.2–11.8 s) because the model is tiny; PyTorch is the fastest at inference (about 1.6 ms per 1,000 windows).

2 RNN Fundamentals

Each term is defined once and keeps its meaning in the rest of the report. Every formula below is checked numerically in `notebook/00_rnn_concepts.ipynb` (against NumPy, finite differences, PyTorch autograd and PyTorch cells). Notation: N samples, T window length, F features per step, H hidden size; $x_t \in \mathbb{R}^F$ input at step t , $h_t \in \mathbb{R}^H$ hidden state, $\hat{y}$ prediction, y target, L loss.

2.1 Sequences as data

A window of a time series is a tensor $X \in \mathbb{R}^{N \times T \times F}$ with a target $y \in \mathbb{R}^N$. For the series $[10, 12, 11, 15, 14, 18]$ and $T = 3$ the windows are $[10, 12, 11] \rightarrow 15$, $[12, 11, 15] \rightarrow 14$, $[11, 15, 14] \rightarrow 18$. *Many-to-one* models read T steps and emit one prediction from the last state (used here); *many-to-many* models emit an output at every step. Because the future must never leak into the past, data are split *chronologically* (train $\rightarrow$ validation $\rightarrow$ test in time order), never at random.

2.2 Operators

- **Matrix product** $(xW)_j = \sum_i x_i W_{ij}$ mixes inputs into new values: $(1, 2) \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix} = (7, 2, 2)$.
- **Element-wise (Hadamard) product** $(a \odot b)_i = a_i b_i$ implements gates: $(0.9, 0.1) \odot (4, 4) = (3.6, 0.4)$.
- $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$, with $\tanh' = 1 - \tanh^2$: $\tanh(1) = 0.7616$, $\tanh(3) = 0.9951$ and $\tanh'(3) = 0.0099$, so a saturated unit passes almost no gradient.
- $\sigma(z) = 1/(1 + e^{-z}) \in (0, 1)$, with $\sigma' = \sigma(1 - \sigma) \leq 0.25$, used for gate values.
- **Softmax** $e^{z_k} / \sum_j e^{z_j}$ turns scores into probabilities (not needed for the regression heads here).

2.3 The recurrent step

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b), \quad h_0 = 0, \quad \hat{y} = h_T W_{hy} + b_y. \quad (1)$$

$W_{xh} \in \mathbb{R}^{F \times H}$, $W_{hh} \in \mathbb{R}^{H \times H}$, $b \in \mathbb{R}^H$: the layer has $FH + H^2 + H$ parameters, *independent of T* , because the same weights are applied at every step (weight sharing). “Unrolling” draws the loop as T copies of the cell. Worked step ($F = 2$, $H = 3$, $h_0 = 0$, $b = 0$, $x_1 = (1, -1)$): $z_1 = (0.3, -0.8, 0.5)$ and $h_1 = \tanh z_1 = (0.2913, -0.6640, 0.4621)$; the notebook reproduces this and matches `torch.nn.RNN` to $3 \cdot 10^{-8}$.

2.4 Back-propagation through time

With $z_t = x_t W_{xh} + h_{t-1} W_{hh} + b$ and $L = \frac{1}{N} \sum (\hat{y} - y)^2$:

$$\frac{\partial L}{\partial h_T} = \frac{2}{N} (\hat{y} - y) W_{hy}^\top, \quad \frac{\partial L}{\partial z_t} = \frac{\partial L}{\partial h_t} \odot (1 - h_t^2), \quad \frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial z_t} W_{hh}^\top, \quad (2)$$

and the shared weights collect a contribution from every step, $\partial L / \partial W_{hh} = \sum_t h_{t-1}^\top \partial L / \partial z_t$ (likewise W_{xh} with x_t , and b with 1). For $T = 3$ the notebook compares these formulas with finite differences (max. abs. error $2.9 \cdot 10^{-10}$) and with PyTorch autograd (error $\leq 2 \cdot 10^{-16}$).

2.5 Vanishing and exploding gradients, clipping

The signal reaching step t is a product of $T - t$ matrices $\text{diag}(1 - h^2)W_{hh}^\top$ and behaves like $(\rho c)^{T-t}$, with ρ the spectral radius of W_{hh} and $c \leq 1$ the typical $\tanh'$. If $\rho c < 1$ the gradient vanishes (a factor 0.5 per step gives $0.5^{20} \approx 10^{-6}$ after 20 steps), if $\rho c > 1$ it explodes ($1.5^{20} \approx 3300$). In the notebook, with $T = 100$, a spectral radius of 0.5 leaves a gradient below 10^{-10} and a radius of 3 inflates it above 10^3 . **Clipping** rescales the gradient by its global norm, $g \leftarrow g \min(1, \tau/\|g\|)$; every run here uses $\tau = 1$ (e.g. norm 50 $\rightarrow$ scale 0.02 $\rightarrow$ norm 1, direction unchanged). Vanishing gradients are addressed architecturally by LSTM/GRU.

2.6 LSTM and GRU

Gates are sigmoid outputs multiplied onto values. The LSTM keeps a hidden state h_t and a cell state c_t :

$$\begin{aligned} i_t &= \sigma(x_t W_i + h_{t-1} U_i + b_i), & f_t &= \sigma(x_t W_f + h_{t-1} U_f + b_f), \\ g_t &= \tanh(x_t W_g + h_{t-1} U_g + b_g), & o_t &= \sigma(x_t W_o + h_{t-1} U_o + b_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned} \quad (3)$$

The update of c_t is *additive*: $\partial c_t / \partial c_{t-1} = f_t$, so with $f_t \approx 1$ the gradient passes unchanged. Example: $f = 0.9$, $c_{t-1} = 2$, $i = 0.5$, $g = 0.4$ gives $c_t = 0.9 \cdot 2 + 0.5 \cdot 0.4 = 2.0$. The GRU merges the two states: $h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$ with reset gate r_t , update gate z_t and candidate $n_t = \tanh(x_t W_n + b_{in} + r_t \odot (h_{t-1} U_n + b_{hn}))$. A vanilla layer has $FH + H^2 + H$ parameters, a GRU three times and an LSTM four times that (one block per gate and candidate). Hand-written LSTM and GRU steps match `LSTMCell` / `GRUCell` to $\leq 6 \cdot 10^{-8}$.

2.7 One layer, three names

Table 2: The same recurrent layer in the three implementations.

	Scratch	Keras SimpleRNN(H)	PyTorch nn.RNN
Input	(N, T, F)	(N, T, F)	(N, T, F) with <code>batch_first=True</code>
W_{xh}	(F, H)	<code>kernel</code> (F, H)	<code>weight_ih_l0</code> (H, F) , transposed
W_{hh}	(H, H)	<code>recurrent_kernel</code>	<code>weight_hh_l0</code> , transposed
Bias	one vector	one vector	two vectors (<code>bias_ih</code> , <code>bias_hh</code>)
Output	last state	last state	all states, <code>take out[:, -1]</code>
Default init	Glorot / orthogonal	Glorot / orthogonal	uniform $\pm 1/\sqrt{H}$
LSTM gate order	–	i, f, c, o	i, f, g, o
GRU gate order	–	z, r, h	r, z, n
Training loop	hand-written BPTT + Adam	<code>fit()</code>	hand-written loop, <code>backward()</code>

3 Datasets and Preprocessing

Both datasets are new in this course. The sizes follow the lecturer's rule that each assignment's dataset be larger than the previous one's (up to 300,000 samples) but not much larger: about 430,000 windows each, with raw files below 100 MB.

3.1 EC: e-commerce service process (Online Retail II)

Source: UCI Online Retail II (Kaggle mashlyn/online-retail-ii-uci), a UK online retailer, 2009-12-01 to 2011-12-09, 1,067,371 invoice lines. Every invoice is an *order*; an invoice whose number starts with **C** is a *cancellation or return*. After removing 34,335 exact duplicate lines, real orders (not **C**, quantity > 0, price > 0) define **604 open days** (the shop never sells on Saturdays and closes over Christmas; those days are dropped, not filled with zero, and the feature **gap** records the calendar days since the previous open day). The 750 products with the most distinct invoices are kept, giving $750 \times (604 - 29) = 432,000$ windows of $T = 28$ open days and $F = 10$ features.

Table 3: EC features (per product and open day).

Feature	Meaning	Transform
units (target)	units ordered	$\log(1 + x)$, z-score
orders, customers	distinct invoices / distinct customer IDs (22 % of lines have no ID)	$\log(1 + x)$, z-score
cancel_units	units in cancellation invoices (service-quality signal)	$\log(1 + x)$, z-score
price	mean unit price	$\log$, z-score
gap	days since previous open day	z-score
dow, doy sin/cos	weekday and day of year on a circle	none

The log transform turns ratios into differences (0, 9 and 99 units become 0, 2.30 and 4.61), so a ten-fold jump looks like a constant step. Task: the last 28 open days $\rightarrow$ units on the next open day, one model for all products. Baselines: *persistence* ($y_t = y_{t-1}$), *seasonal naive* ($y_t = y_{t-6}$, one open-day week) and the train mean.

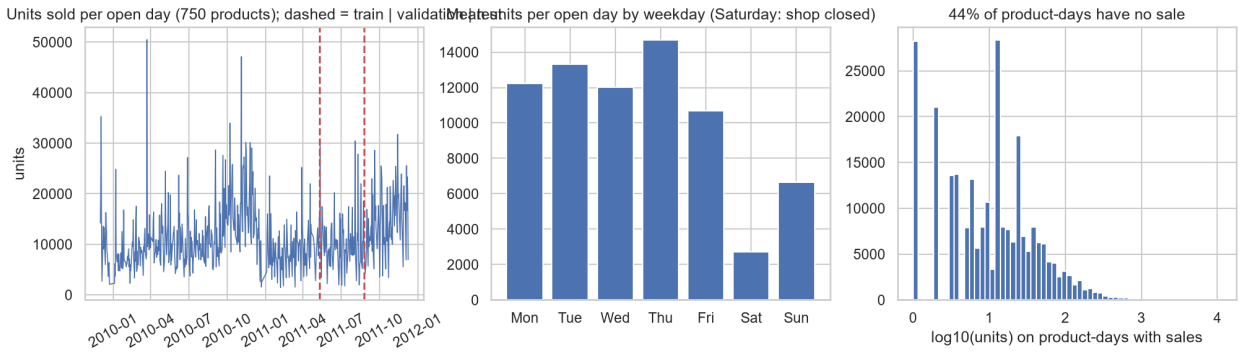


Figure 1: EC: total units per open day (dashed lines: train | validation | test), mean units by weekday (Saturday: shop closed) and the distribution of units per product-day. A large share of product-days has no sale at all.

3.2 ST: S&P 500 stock prices

Source: Kaggle camnugent/sandp500, all_stocks_5yr.csv, 619,040 rows, 505 tickers, 2013-02-08 to 2018-02-07. 470 tickers have the full 1,259-day history; the 350 with the highest mean dollar volume (close $\times$ volume) are kept, giving $350 \times (1258 - 30) = 429,800$ windows of $T = 30$ days and $F = 4$ features. Missing open/high/low values (a handful) are replaced by the day's close.

Table 4: ST features (per stock and day).

Feature	Meaning	Transform
ret (target)	$\ln(\text{close}_t / \text{close}_{t-1})$; +1 % $\rightarrow$ 0.00995, -1 % $\rightarrow$ -0.01005	z-score
hl_range	(high - low)/close, intraday volatility	z-score
oc_ret	$\ln(\text{close}/\text{open})$	z-score
vol_z	$\ln(1 + \text{volume})$	z-score per ticker

Task: the last 30 days $\rightarrow$ log return on the next day, one pooled model for all stocks. Baselines: zero return, previous-day return, always “up” and the train mean. The EDA (Figure 2) shows why the task is hard: returns have excess kurtosis 54.2, lag-1 autocorrelation -0.005 , while *squared* returns have lag-1 autocorrelation 0.094: volatility clusters, direction does not.

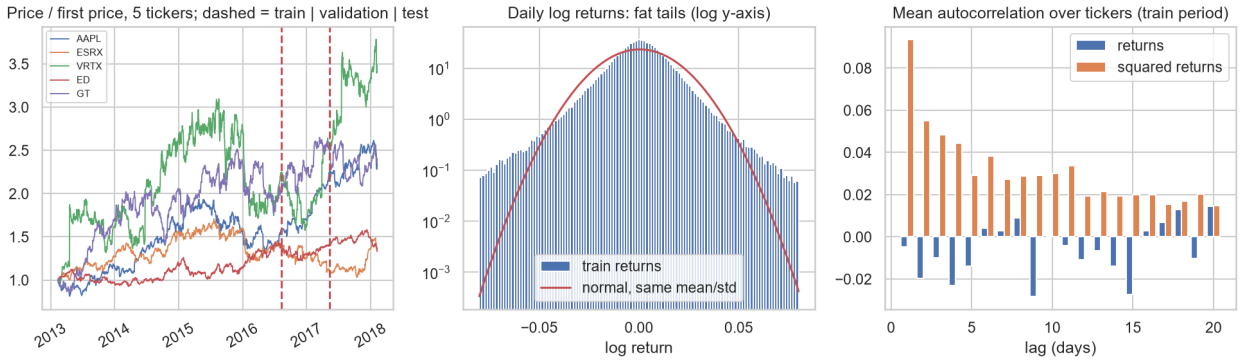


Figure 2: ST: normalised prices of five stocks with the split dates, heavy-tailed return distribution against a normal curve (log axis), and mean autocorrelation of returns and squared returns.

3.3 Splits, scaling and metrics

- **Chronological split by target day:** first 70 % of days train, next 15 % validation, last 15 % test (EC: train $<$ 2011-05-10 $\leq$ validation $<$ 2011-08-25 $\leq$ test; ST: train $<$ 2016-08-09 $\leq$ validation $<$ 2017-05-10 $\leq$ test). Window counts: EC 295,500 / 68,250 / 68,250, ST 297,500 / 66,150 / 66,150. A window may look back into an earlier split, but its target never leaves its own split.
- **Scaling** (mean and standard deviation) is computed on the train period only.
- **Metrics:** RMSE and MAE in original units (EC: units sold after undoing the transforms; ST: log-return), and out-of-sample $R^2 = 1 - \sum(y - \hat{y})^2 / \sum y^2$ in z-space, i.e. against the train-mean predictor ($R^2 < 0$ means worse than that predictor). ST adds *directional accuracy* (sign of prediction equals sign of return) and the *information coefficient* (IC, correlation of prediction and truth).

4 Models and Fairness Rules

Core model (all three implementations): input $(N, T, F) \rightarrow$ one vanilla RNN layer ($H = 32$, tanh) $\rightarrow$ last state $h_T \rightarrow$ linear layer with one output. Parameters $= FH + H^2 + H + H + 1$: **1,409** for EC ($F = 10$) and **1,217** for ST ($F = 4$). The loss is MSE on the z-scored target. **Extras:** LSTM(32) and GRU(32) in Keras and PyTorch with framework-default initialisation.

Table 5: Settings shared by every run.

Hidden size	32
Optimizer	Adam, learning rate 10^{-3} , $\epsilon = 10^{-7}$
Batch size	128 (shuffled within train)
Epochs	at most 30; early stopping, patience 5 on validation loss, best weights re-stored
Gradient clipping	global norm 1.0
Initial weights (core RNN)	scratch initialisation copied into Keras and PyTorch
Seed	42
Metric of record	test period, computed once after training

Same weights. The scratch model draws Glorot-uniform W_{xh} , orthogonal W_{hh} and zero biases. The same arrays are copied into Keras (`set_weights`) and PyTorch (transposed). PyTorch has a second bias, which is zeroed and frozen so that the parameter counts are equal; this is asserted in code. Because the frameworks shuffle with their own generators and Adam’s ϵ is placed slightly differently in each, training is not bit-identical across implementations; only the starting point is.

Naming. Run ID = <dataset>-<implementation>-<model> with dataset EC/ST, implementation SC (scratch), TF (Keras), PT (PyTorch) and model RNN/LSTM/GRU. The same ID names the result file, the figures and the table rows.

5 Implementations

5.1 Scratch (NumPy)

The class `ScratchRNN` stores the five arrays $W_{xh}, W_{hh}, b, W_{hy}, b_y$. The forward pass precomputes XW_{xh} for all steps at once and loops only over time; the backward pass loops from $t = T$ to 1 applying Equations (2)–(3), accumulating the shared-weight gradients, and also returns $\|\partial L / \partial h_t\|$ for diagnostics. Adam and global-norm clipping are written by hand.

Gradient check. Before training, every parameter of the scratch model is checked against central finite differences ($\epsilon = 10^{-6}$, float64, 8 real windows). The error is measured per parameter as $\max |a - n| / \max |a|$:

Table 6: Gradient check: BPTT versus finite differences (all entries below the 10^{-5} threshold).

Parameter	EC (error)	ST (error)
W_{xh}	$3.8 \cdot 10^{-10}$	$8.6 \cdot 10^{-10}$
W_{hh}	$9.3 \cdot 10^{-10}$	$8.3 \cdot 10^{-10}$
b	$4.4 \cdot 10^{-10}$	$1.6 \cdot 10^{-9}$
W_{hy}	$2.1 \cdot 10^{-10}$	$3.4 \cdot 10^{-10}$
b_y	$5.0 \cdot 10^{-12}$	$1.9 \cdot 10^{-10}$

(A first version used the element-wise relative error $|a - n| / (|a| + |n|)$; one almost-zero entry of W_{hh} on ST then showed $1.3 \cdot 10^{-5}$ purely from finite-difference round-off, so the check was changed to the scale-aware form above.)

5.2 Keras

`Sequential([Input((T,F)), SimpleRNN(32), Dense(1)])`, compiled with Adam (`global_clipnorm=1.0`, $\epsilon = 10^{-7}$) and MSE; `fit` runs shuffled mini-batches with `EarlyStopping(restore_best_weights=True)`. LSTM and GRU replace the first layer.

5.3 PyTorch

A module with `nn.RNN(F, 32, batch_first=True)` and `nn.Linear(32, 1)` applied to `out[:, -1]`; a hand-written loop does `zero_grad` $\rightarrow$ `forward` $\rightarrow$ `backward` $\rightarrow$ `clip_grad_norm` $\rightarrow$ `step` and restores the best state. LSTM and GRU use `nn.LSTM/nn.GRU`. These keep two biases, hence their counts differ slightly from Keras’s LSTM (by $4H = 128$); GRU matches because Keras uses `reset_after=True`.

5.4 Equivalence check

With the same weights, the three forward passes on 64 validation windows differ by at most $4.8 \cdot 10^{-7}$ (EC: scratch–Keras $4.8 \cdot 10^{-7}$, scratch–PyTorch $4.8 \cdot 10^{-7}$, Keras–PyTorch $6.0 \cdot 10^{-7}$; ST: 4.8, 3.6, $3.3 \cdot 10^{-7}$), which is float32 round-off, and the parameter counts are identical (1,409 and 1,217).

6 Results on EC (e-commerce service process)

Table 7: EC, test period (68,250 windows). RMSE and MAE in units sold; R^2 on the log scale; epochs run (best epoch); s/epoch = median over epochs 2+; ms/1k = inference time per 1,000 windows.

Run	RMSE	MAE	R^2	params	epochs (best)	s/epoch	ms/1k
<i>persistence</i> ($y[t]=y[t-1]$)	82.68	23.91	-0.044	–	–	–	–
<i>seasonal naive</i> ($y[t]=y[t-6]$)	81.06	23.19	-0.030	–	–	–	–
<i>train mean</i>	67.88	19.26	0.000	–	–	–	–
EC-SC-RNN	62.51	15.90	0.439	1,409	12 (7)	3.9	5.54
EC-TF-RNN	63.01	16.02	0.438	1,409	22 (17)	6.2	8.02
EC-PT-RNN	62.84	16.00	0.439	1,409	14 (9)	11.2	1.54
EC-TF-LSTM	62.44	15.93	0.446	5,537	12 (7)	14.6	16.90
EC-PT-LSTM	62.03	15.83	0.447	5,665	18 (13)	22.0	4.14
EC-TF-GRU	62.41	15.89	0.449	4,257	16 (11)	16.4	15.05
EC-PT-GRU	62.24	15.87	0.448	4,257	15 (10)	28.4	5.12

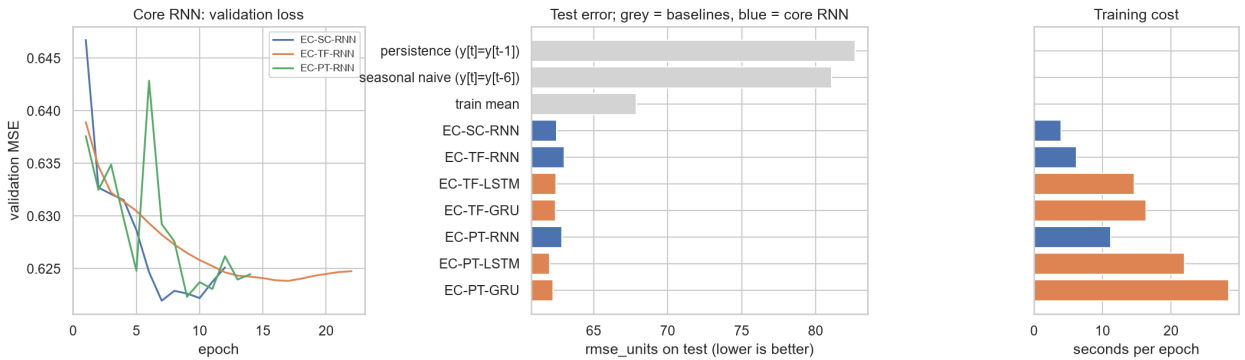


Figure 3: EC: validation loss of the three core RNNs, test RMSE of all runs against the baselines (grey), and time per epoch.

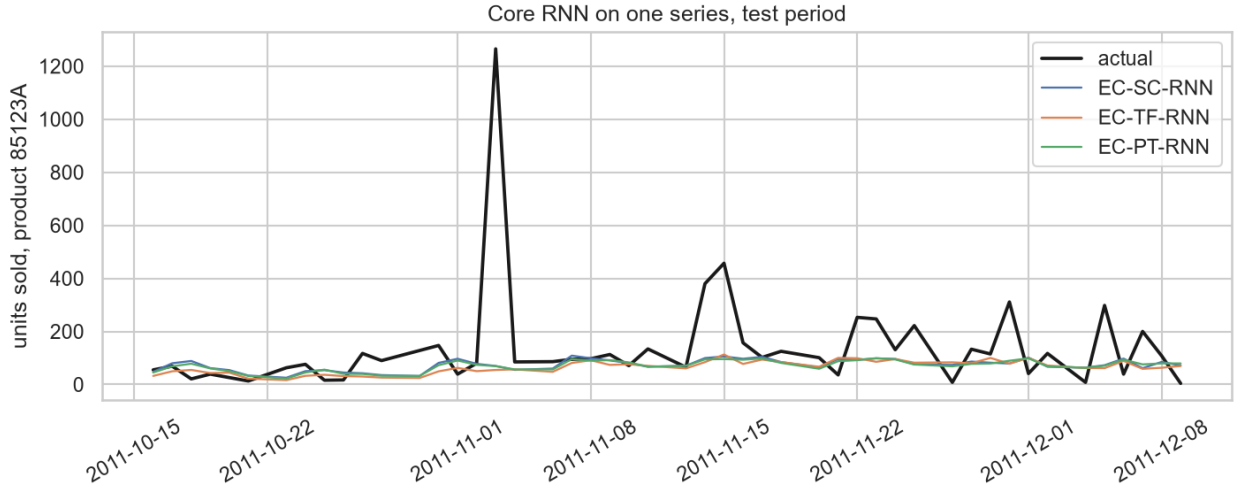


Figure 4: EC: forecasts of the three core RNNs for the most-ordered product over the last 48 test days. The models follow the level and the weekly rhythm but, as with any regression to the mean, not the one-off spike.

Discussion.

- **RNNs beat every baseline.** Persistence (82.7) and seasonal naive (81.1) are worse than the train mean (67.9) because demand is intermittent and spiky; the RNNs (62.0–63.0) are 8–9 % below the mean and 24–25 % below persistence in RMSE.
- **Implementations agree.** Core RNN RMSE: scratch 62.51, Keras 63.01, PyTorch 62.84, with R^2 0.439 / 0.438 / 0.439. The validation curves (left panel of the comparison figure) have different shapes because of different batch order; Keras stops at epoch 22 (best 17), scratch at 12 (best 7), PyTorch at 14 (best 9).
- **Gates:** LSTM/GRU give 62.0–62.4 (best EC-PT-LSTM, 62.03) against 62.5–63.0 for the vanilla RNN, i.e. 0.5–1.5 % better, for 3–4× the parameters and 2–3.5× the time per epoch.
- **Memory.** The gradient reaching the first of the 28 days is 0.1 % of the last-step gradient at initialisation and 13.7 % after training: training lengthened the effective memory.
- **Cost.** Vanilla RNN, seconds per epoch: scratch 3.9, Keras 6.2, PyTorch 11.2; inference per 1,000 windows: 5.5, 8.0 and 1.5 ms.

7 Results on ST (stock returns)

Table 8: ST, test period (66,150 windows). RMSE in log-return; R^2 against the train-mean predictor; dir. acc. = directional accuracy; IC = information coefficient. The “always up” rate on this test period is 0.531 (the train mean is positive, so it predicts “up” everywhere; the zero predictors have no sign).

Run	RMSE	R^2	dir. acc.	IC	params	epochs (best)	s/epoch	ms/1k
<i>zero return</i>	0.0157	-0.001	—	—	—	—	—	—
<i>always up (+1e-6)</i>	0.0157	-0.001	—	—	—	—	—	—
<i>previous-day return</i>	0.0223	-1.016	0.492	-0.011	—	—	—	—
<i>train mean</i>	0.0157	0.000	0.531	—	—	—	—	—
ST-SC-RNN	0.0159	-0.017	0.508	-0.015	1,217	7 (2)	4.2	6.19
ST-TF-RNN	0.0159	-0.017	0.483	-0.021	1,217	6 (1)	6.5	7.36
ST-PT-RNN	0.0158	-0.016	0.496	-0.027	1,217	6 (1)	11.8	1.63
ST-TF-LSTM	0.0158	-0.014	0.483	-0.021	4,769	6 (1)	14.9	18.19
ST-PT-LSTM	0.0158	-0.012	0.493	-0.022	4,897	6 (1)	20.3	3.95
ST-TF-GRU	0.0158	-0.015	0.483	-0.023	3,681	6 (1)	17.0	16.60
ST-PT-GRU	0.0158	-0.013	0.485	-0.028	3,681	6 (1)	29.7	5.52

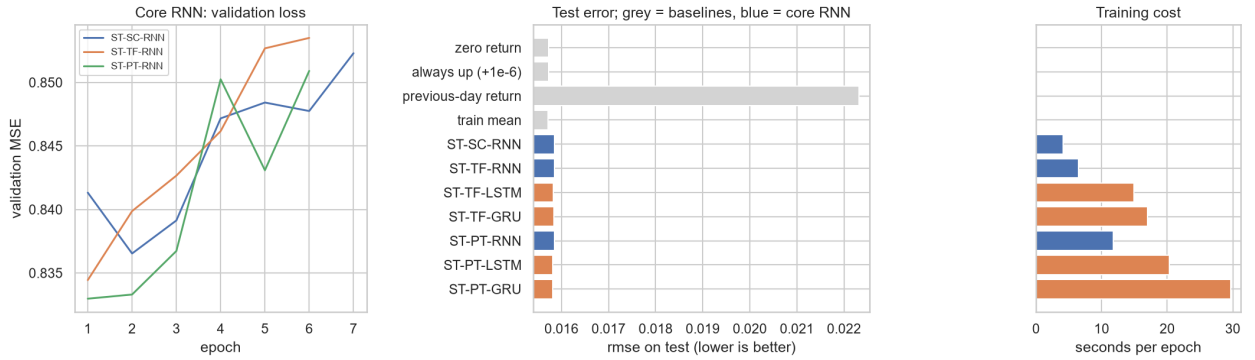


Figure 5: ST: validation loss of the core RNNs, test RMSE of all runs against the baselines (grey), and time per epoch. The bar scale is truncated, so the differences look larger than they are (all seven runs are within 0.3% of each other).

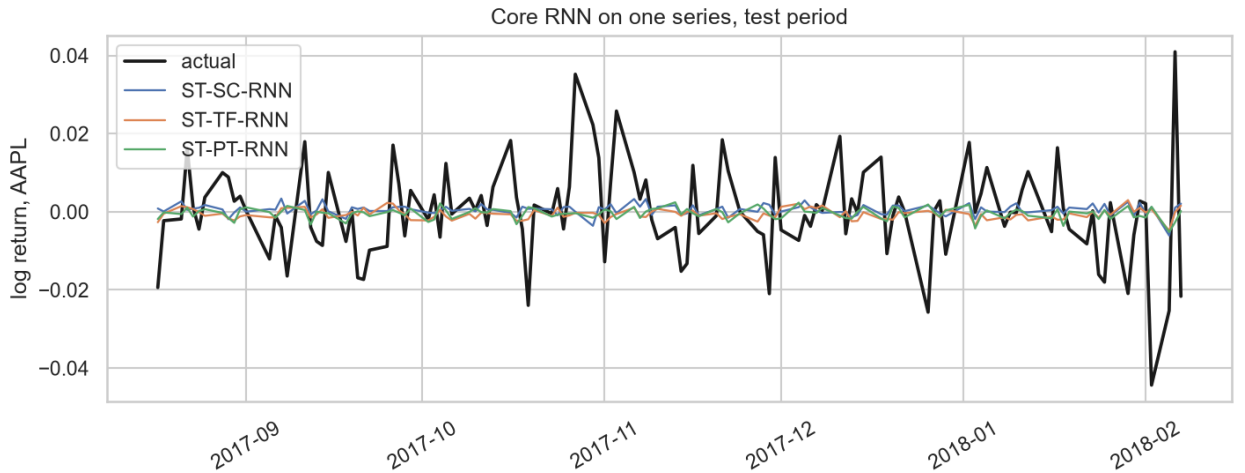


Figure 6: ST: predicted and actual daily log return for AAPL over the last 120 test days. The predictions hug zero: the models learn that they cannot predict.

Discussion.

- **No model beats the trivial forecast.** RMSE 0.0158–0.0159 against 0.0157 for zero return; R^2 -0.012 to -0.017 ; IC -0.015 to -0.028 . Predicting yesterday’s return is far worse (RMSE 0.0223, $R^2 = -1.02$).
- **Direction is not predictable.** Directional accuracy is 48.3–50.8%, below the 53.1% of always saying “up” in a rising test period. Every run reaches its best validation loss at epoch 1–2 and then only fits noise.
- **Why.** Returns are almost uncorrelated with their past (lag-1 autocorrelation -0.005) and heavy-tailed (excess kurtosis 54), so MSE is dominated by a few extreme days; only the *size* of moves (volatility) is predictable and that is not what an MSE-trained point forecast of the *sign* uses.
- **Implementations still agree:** RMSE 0.0159 / 0.0159 / 0.0158, R^2 -0.017 / -0.017 / -0.016 . LSTM/GRU do not help (R^2 -0.012 to -0.015) and cost $2\text{--}3.5\times$ more per epoch.
- **Price view.** Turning predicted returns into prices ($\hat{P}_{t+1} = P_t e^{\hat{r}}$) makes every model look excellent because yesterday’s price carries almost all the information (Figure in the notebook, `ST_price_demo.png`); the return-space metrics are the honest ones.

8 Cross-Dataset Discussion

- **Same code, opposite outcomes.** The architecture, settings and implementations are identical on EC and ST, yet EC is learnable ($R^2 \approx 0.44$) and ST is not ($R^2 < 0$). The difference lies in the data: weekly and yearly rhythm, order and cancellation counts carry information about next-day demand; daily stock returns carry almost none about the next return. A good report on ST is therefore a negative result with the right baselines.
- **Scratch = Keras = PyTorch in function.** On both datasets the three core RNNs start from the same function and end within 0.8% (EC) and 0.03% (ST) of each other; the abstraction level changes the amount of code and the speed, not the learned model.
- **Gates** improve a series with long, regular patterns a little (EC) and do nothing for noise (ST).
- **Speed.** On CPU with $H = 32$ the NumPy loop is the fastest to train; PyTorch pays Python overhead per mini-batch in its hand-written loop (and per-op overhead on tiny tensors), Keras sits in between; PyTorch is the fastest for inference.

9 Conclusion and Limitations

The assignment covered the concepts of an RNN (sequence tensors, operators, recurrence, BPTT, vanishing gradients and clipping, LSTM/GRU gates), two time-related datasets of about 430,000 windows each, and three implementations of the same RNN (scratch, Keras, PyTorch) that were verified to be the same function (forward-pass agreement $< 10^{-6}$, gradient check $< 10^{-9}$, equal parameter counts) and compared fairly. On e-commerce order flow the RNN beats all baselines; on stock returns it correctly learns that there is nothing to predict.

Limitations. One seed and one chronological split per dataset: differences below about 1% between runs should not be read as a ranking. Hidden size and window length were fixed, not tuned; the planned window-length ablation was not run. The ST pool contains survivors of the index (stocks with complete histories), and results describe 2013–2018 only. Timings are CPU-only (TensorFlow has no GPU build on native Windows), so the speed ranking may change on a GPU.

The scratch implementation covers the vanilla RNN only; LSTM and GRU exist in Keras and PyTorch.

Reproducibility. `assignment_06/` contains `dataset/README.md` (download links), the notebooks `00_rnn_concepts`, `ec_retail` and `st_sp500` (each runs top to bottom in about 30 minutes on a CPU; `A6_SMOKE=1` gives a quick test), `notebook/results/*.json` (metrics and training history of every run) and `report/make_tables.py`, which builds the tables of this report from those files. Software: Python 3.13, TensorFlow 2.21 / Keras 3.15, PyTorch 2.14, NumPy 2.5.